# Adaptation-Aware Model Selection for Few-Shot Digital Twin Instantiation

ARTICLE INFO

*Keywords*:
digital twin
few-shot adaptation
model instantiation
architecture selection
model deployment limits
heterogeneous computing centers

ABSTRACT

As an important modeling tool for managing computing centers, digital twin (DT) platforms reuse pre-trained DT models from existing centers to reduce the cost of DT instantiation in large-scale computing networks. Target-center heterogeneity in resource configurations and workload characteristics complicates direct model reuse when target adaptation uses a small support set. However, existing few-shot adaptation methods typically retain a predefined source architecture and therefore do not directly address target-specific model selection for heterogeneous centers under per-model deployment limits. To address this challenge, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI), an adaptation-aware model-selection algorithm for DT instantiation with few-shot target adaptation under deployment limits. MSA-DTI constructs a reusable candidate bank of source-trained models with diverse architectures, filters candidates that violate the operation-count and parameter-count limits, and adapts the remaining candidates using the same target support set and update budget. It then compares post-adaptation performance on a separate labeled validation set and replaces a predefined reference model only when the best-performing alternative achieves a calibrated relative reduction in validation loss. On the controlled synthetic benchmark, MSA-DTI achieves the lowest mean squared error (MSE) among the evaluated methods and reduces case-wise MSE by 14.60% on average relative to the pre-training-and-fine-tuning baseline, while satisfying both deployment limits. In a separately source-trained and calibrated Bitbrains real-workload evaluation, we observe a 32.70% mean case-relative test-MSE reduction versus PT+FT (95% VM-cluster CI [7.67%,51.25%]), with increased target-side adaptation cost.

## 1. Introduction

As an important modeling tool for managing computing centers, a shared digital twin (DT) platform maintains center-specific models for monitoring, prediction, analysis, and decision making [1–4]. In large-scale computing networks, reusing and adapting pre-trained DT models instead of constructing each target model from scratch reduces DT-instantiation cost [5, 6]. Each pre-trained model comprises an architecture and source-trained parameters. The architecture determines the model structure and deployment-related complexity, whereas the parameters provide the initialization for target adaptation. For a new or reconfigured center, differences in resource configurations, monitored-variable availability, workload patterns, and data distributions complicate direct reuse of a source model, while target adaptation uses a small support set. The source architecture is therefore not necessarily suitable for the target center. In addition, large-scale DT platforms impose per-model resource budgets, represented here by limits on estimated operation count per inference and parameter count. Target-center instantiation therefore requires selecting a suitable reusable model and adapting its parameters from the small support set while satisfying both deployment limits. Figure 1 illustrates this setting.

Existing few-shot adaptation methods reuse pre-trained models [7, 8] and have also been applied to DT modeling [6]. However, these methods typically retain the source architecture and therefore do not directly address target-specific model selection for heterogeneous target centers under deployment limits. Deployment-aware architecture-selection methods consider model performance together with deployment cost [9–11], but do not directly address few-shot adaptation of reusable pre-trained models. Methods that jointly consider architecture decisions and few-shot adaptation [12–14] typically rely on target-side architecture or adaptation-structure search. The remaining challenge is therefore to select a suitable reusable pre-trained DT model under per-model deployment limits and adapt its parameters from a small target support set.

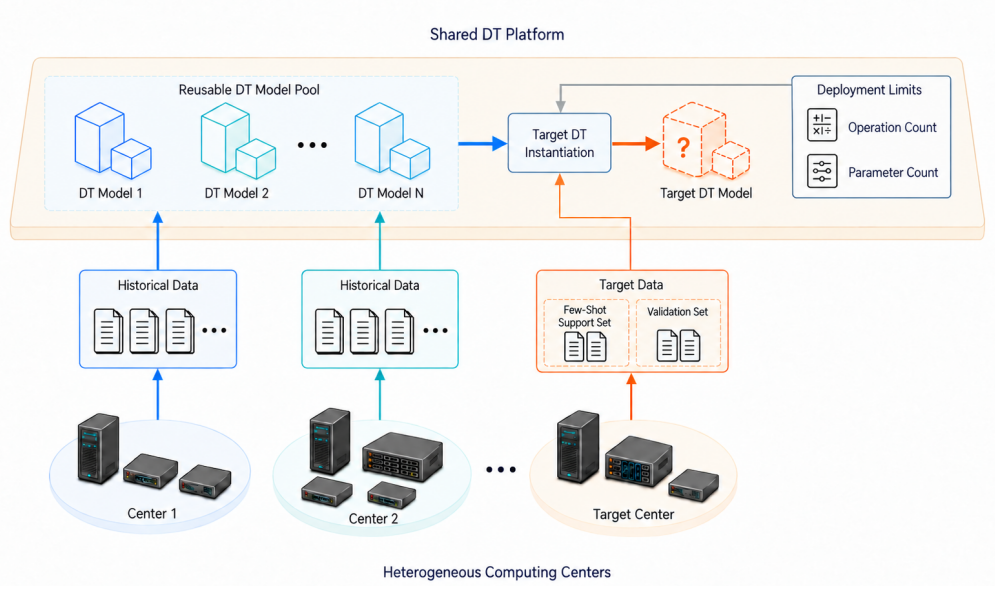



Figure 1: System setting for target-specific DT instantiation.

To address this challenge, we propose Model Selection and Adaptation for Digital Twin Instantiation (MSA-DTI), an adaptation-aware model-selection algorithm for DT instantiation with few-shot target adaptation under deployment limits. MSA-DTI first constructs a reusable candidate bank of source-trained models with diverse architectures. For a target center, it filters candidates that violate the deployment limits and adapts the remaining feasible candidates under a common adaptation budget. It then compares the adapted candidates using validation performance and retains a predefined reference model unless the best-performing alternative achieves a sufficient reduction in validation loss. Thus, model selection is based on post-adaptation performance without target-side architecture or adaptation-structure search.

The main contributions are summarized below.

- We formulate DT instantiation as jointly selecting among reusable source-trained models with different architectures and adapting the selected model's parameters from a small target support set, subject to hard per-model operation-count and parameter-count limits.
- We propose a feasibility-first, adaptation-aware model-selection strategy that filters out infeasible candidates before target adaptation and compares the remaining candidates only after adapting them using the same target support set and update budget.
- We introduce a reference-model replacement rule with a relative-improvement threshold calibrated on disjoint development data. An adapted alternative replaces the predefined reference only when it achieves the calibrated relative reduction in validation loss.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 defines the system model and problem. Section 4 presents MSA-DTI. Section 5 reports the experiments, and Section 6 concludes the paper.

## 2. Related Work

### 2.1. Digital Twin Modeling and Platform Deployment

In communication and computing systems, digital twins represent system states and services while supporting model updates [5, 15–17]. At the deployment level, related work considers service orchestration and computation offloading under resource constraints [18, 19]. However, DT modeling studies typically assume a predefined model structure, whereas platform-level research focuses on resource decisions. Neither line of work directly addresses architecture selection together with few-shot target adaptation under deployment limits.

### 2.2. Limited-Data Model Adaptation

Limited-data methods leverage source or prior knowledge for target-model adaptation [7, 8, 20]. Recent approaches improve cross-task and cross-domain adaptation through context modeling and transferable representations [21–24]. Limited-data learning has also been applied to DT construction and workload prediction [6, 25]. Most parameter-adaptation methods operate with a fixed architecture. For heterogeneous target centers, a fixed architecture is not necessarily suitable, motivating selection among reusable source-trained models.

A direct neighboring line of work selects pre-trained models from a finite model hub for a target task. LogME assesses model suitability before target fine-tuning and supports regression, whereas model-hub ranking and tuning further study how pre-trained candidates can be exploited [28, 29]. In contrast, the present study combines two hard per-model deployment limits with common-budget few-shot adaptation of feasible source-trained candidates, post-adaptation validation comparison, and a calibrated option to retain an explicit reference. The constituent ideas of pre-trained model ranking and model reuse are not claimed as new.

Model selection introduces a related challenge because candidate rankings based on validation estimates are subject to uncertainty [26, 27]. This motivates a conservative decision rule when adapted reusable models are compared on target validation data.

### 2.3. Deployment-Aware Architecture Selection

Deployment-aware neural architecture search evaluates model performance together with deployment cost [9–11]. Few-shot, zero-shot, and training-free NAS variants reduce the cost of architecture evaluation during search [30–32]. In Ref. [30], "few-shot" denotes evaluation with several sub-supernets rather than adaptation from a small target support set.

Among meta-learning-based NAS methods, T-NAS learns transferable architecture knowledge for new-task adaptation [13]. H-Meta-NAS further incorporates hardware constraints into task-adaptive search [12]. Neural fine-tuning search instead starts from a pre-trained backbone and searches over adaptation structures, including which components to fine-tune or augment [14]. Together, these methods extend task adaptation beyond parameter updates to architecture or adaptation-structure decisions.

To address target-specific model selection under deployment limits without target-stage search for new architectures or adaptation structures, MSA-DTI selects among a finite bank of separately source-trained reusable models using deployment feasibility and post-adaptation performance.

## 3. System Model and Problem Formulation

### 3.1. System Setting and Target Model

We consider a shared DT platform that maintains center-specific models for heterogeneous computing centers.

Let $c$ denote a new or reconfigured target center. To enable model reuse across centers with different monitored-variable availability, all observations follow a common metric schema, with unavailable variables represented by masks rather than omitted. Let $\mathcal{X}$ denote the common input space defined by this metric schema, and let $\mathcal{Y}$ denote the output space of the DT task. Let $\mathcal{A}$ denote the set of model architectures available to the shared DT platform. For $a \in \mathcal{A}$, let $\Theta(a)$ denote the trainable-parameter space associated with architecture $a$, and let $\theta \in \Theta(a)$ denote a particular parameter setting. A model is represented as

$$f_{a,\theta} : \mathcal{X} \to \mathcal{Y}, \qquad a \in \mathcal{A},\ \theta \in \Theta(a). \tag{1}$$

The DT model instantiated for center $c$ is represented as

$$\mathrm{DT}_c = \left(a_c, \theta_c\right), \qquad a_c \in \mathcal{A},\ \theta_c \in \Theta(a_c), \tag{2}$$

where $a_c$ and $\theta_c$ denote the target architecture and target-specific parameters, respectively. During instantiation, the available target data are represented by a support set and a validation set.

$$\begin{aligned} \mathcal{D}_c^{\mathrm{sup}} &= \left\{ (x_{c,i}^{\mathrm{sup}}, y_{c,i}^{\mathrm{sup}}) \right\}_{i=1}^{K_c}, \\ \mathcal{D}_c^{\mathrm{val}} &= \left\{ (x_{c,j}^{\mathrm{val}}, y_{c,j}^{\mathrm{val}}) \right\}_{j=1}^{V_c}, \\ \mathcal{D}_c^{\mathrm{tar}} &= \mathcal{D}_c^{\mathrm{sup}} \cup \mathcal{D}_c^{\mathrm{val}}, \qquad \mathcal{D}_c^{\mathrm{sup}} \cap \mathcal{D}_c^{\mathrm{val}} = \varnothing. \end{aligned} \tag{3}$$

In these sets, $(x_{c,i}^{\mathrm{sup}}, y_{c,i}^{\mathrm{sup}})$ and $(x_{c,j}^{\mathrm{val}}, y_{c,j}^{\mathrm{val}})$ denote the $i$th support and $j$th validation input-output pairs, respectively. Their union $\mathcal{D}_c^{\mathrm{tar}}$ denotes the target data available during instantiation. $K_c$ denotes the number of target support samples used for parameter adaptation, whereas $V_c$ denotes the number of validation samples used for post-adaptation candidate comparison. The present study concerns determining $(a_c, \theta_c)$ during target-center instantiation. Subsequent model updates are outside its scope.

### 3.2. Reusable Models and Deployment Limits

The shared DT platform maintains source-trained models learned from historical data of existing centers. Each reusable model consists of a model architecture and its source-trained parameter initialization and serves as a candidate for target-center instantiation. Let the finite set of $N$ reusable source-trained models be

$$\mathcal{Q} = \{q_n\}_{n=1}^{N}, \qquad q_n = (a_n, \theta_n^0), \tag{4}$$

where $q_n$ denotes the $n$th reusable source-trained model, $a_n \in \mathcal{A}$ denotes its architecture, and $\theta_n^0 \in \Theta(a_n)$ denotes its source-trained initialization. The architectures represented in $\mathcal{Q}$ form the set $\mathcal{A}_{\mathcal{Q}} = \{a_n\}_{n=1}^{N} \subseteq \mathcal{A}$.

For target center $c$, the platform specifies two per-model deployment limits. They are a maximum estimated operation count for one batch-size-one forward pass and a maximum number of trainable parameters. Let $B_c^F$ and $B_c^P$ denote these limits, respectively. For $a \in \mathcal{A}_{\mathcal{Q}}$, let $F_c(a)$ denote the estimated operation count under the fixed counting rule, and let $P_c(a)$ denote the exact number of trainable parameters. In this formulation, parameter adaptation changes only parameter values while keeping the architecture fixed. Therefore, $F_c(a)$ and $P_c(a)$ remain unchanged.

For center $c$, the deployment-feasible reusable model set is

$$\mathcal{Q}_c^{\mathrm{fea}} = \left\{ q_n \in \mathcal{Q} \;\middle|\; F_c(a_n) \le B_c^F,\ P_c(a_n) \le B_c^P \right\}. \tag{5}$$

Both deployment limits are modeled as hard constraints rather than penalty terms.

### 3.3. DT Model Instantiation Problem

Let $\mathbb{P}_c$ denote the target-center data distribution over $\mathcal{X} \times \mathcal{Y}$, and let $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}_{\ge 0}$ denote the task loss function. The target risk is defined as

$$R_c(a, \theta) = \mathbb{E}_{(x,y)\sim\mathbb{P}_c}\left[\ell(f_{a,\theta}(x), y)\right]. \tag{6}$$

Let $\mathcal{U}_c$ denote a prescribed target-adaptation operator. For each reusable model $q_n = (a_n, \theta_n^0)$, $\mathcal{U}_c(q_n, \mathcal{D}_c^{\mathrm{sup}}) \in \Theta(a_n)$ denotes the target-specific parameters obtained by adapting $q_n$ using the target support set. An ideal reusable-model decision for center $c$ is defined as

$$\begin{aligned} n_c^{\dagger} \in \underset{n\in\{1,\ldots,N\}}{\arg\min} \quad & R_c\big(a_n, \mathcal{U}_c(q_n, \mathcal{D}_c^{\mathrm{sup}})\big) \\ \text{s.t.} \quad & F_c(a_n) \le B_c^F, \\ & P_c(a_n) \le B_c^P. \end{aligned} \tag{7}$$

Here, $n_c^{\dagger}$ denotes the reusable-model index selected under the constrained target-risk criterion in Eq. (7). The corresponding DT model is $\left(a_c^{\dagger}, \theta_c^{\dagger}\right) = \left(a_{n_c^{\dagger}}, \mathcal{U}_c(q_{n_c^{\dagger}}, \mathcal{D}_c^{\mathrm{sup}})\right)$. The target distribution $\mathbb{P}_c$ is unavailable during instantiation. The target risk $R_c$ therefore cannot be evaluated directly. The decision must instead be based on the support and validation data in Eq. (3).

## 4. The Proposed MSA-DTI Algorithm

### 4.1. Overview

Figure 2 summarizes MSA-DTI for the instantiation problem in Eq. (7). The method contains an offline preparation stage and a target-side instantiation stage. Offline preparation constructs the reusable candidate bank $\mathcal{Q}$, designates the reference candidate $q^{\mathrm{ref}}$, and calibrates the relative margin $\tau$ on development data that are disjoint from final target evaluation. For a target center, the target-side stage filters candidates by the two deployment limits, adapts the feasible candidates using the same support set and update budget, and compares their post-adaptation performance

on the validation set. The best alternative replaces the adapted reference only when it satisfies the calibrated relative-improvement requirement. The output is the target-specific DT model $\mathrm{DT}_c$.

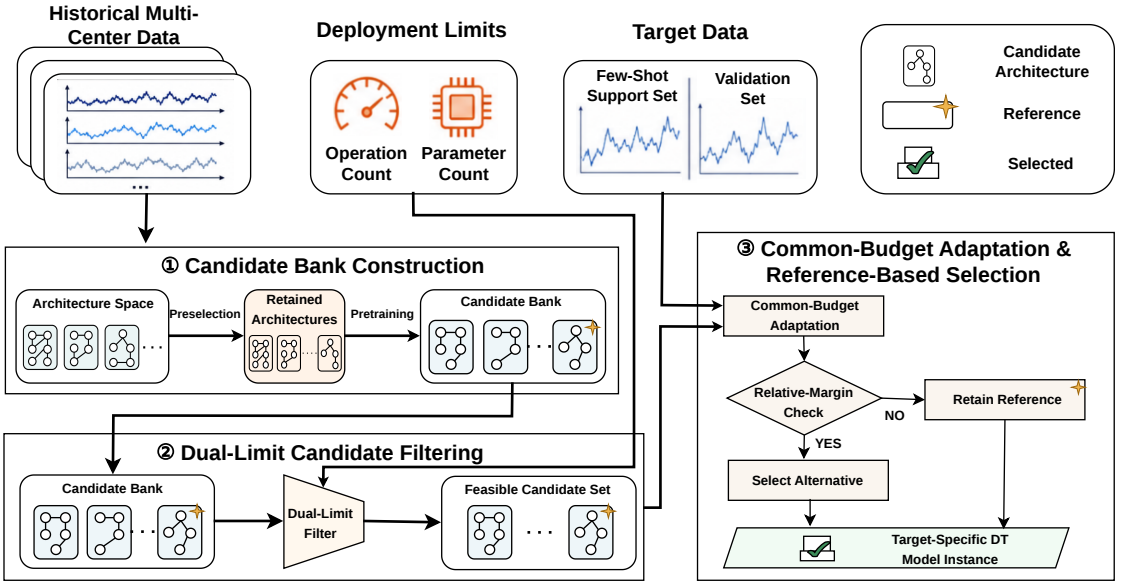



Figure 2: Overall workflow of MSA-DTI.

## 4.2. Candidate Bank Construction

Before constructing the final source-trained candidate bank, MSA-DTI performs architecture preselection to identify the architectures to retain. This step uses a dedicated pool of development target-center cases that is disjoint from the data used for margin calibration and final target evaluation. A reference architecture $a^{\mathrm{ref}} \in \mathcal{A}$ is fixed before preselection and retained regardless of the preselection outcome. Each preselection case contains three disjoint subsets. The support subset is used for adaptation, the validation subset is used to select an alternative architecture, and the check subset provides preselection evidence. For each preselection case, the reference architecture must satisfy the assigned deployment limits, and only feasible alternative architectures are included in the comparison. During preselection, the reference and each feasible alternative are adapted from fixed source-trained initializations on the support subset before validation and check losses are evaluated.

For each alternative architecture, two evidence counts are accumulated across the preselection cases. A *check-oracle win* is assigned to the feasible alternative architecture with the lowest check-set loss if its loss is lower than that of the adapted reference. A *validation-selected positive check win* is assigned to the feasible alternative architecture selected on the validation subset if its check-set loss is lower than that of the adapted reference. The architecture is retained if at least one of the two counts reaches a preset threshold.

Let $\mathcal{D}_{\mathrm{src}}$ denote the combined historical data from the existing centers. For each retained architecture $a_q$, source pretraining on $\mathcal{D}_{\mathrm{src}}$ produces an initialization $\theta_q^0$, forming a candidate $q = (a_q, \theta_q^0) \in \mathcal{Q}$. For a retained architecture with multiple source-trained initializations, each initialization defines a distinct candidate. One candidate associated with the reference architecture $a^{\mathrm{ref}}$ is designated as $q^{\mathrm{ref}}$. Figure 3 summarizes the retained architectures and resulting candidate bank. The candidate bank and reference candidate are fixed before margin calibration and final target evaluation. This stage outputs $\mathcal{Q}$ and $q^{\mathrm{ref}}$ for target-side instantiation.

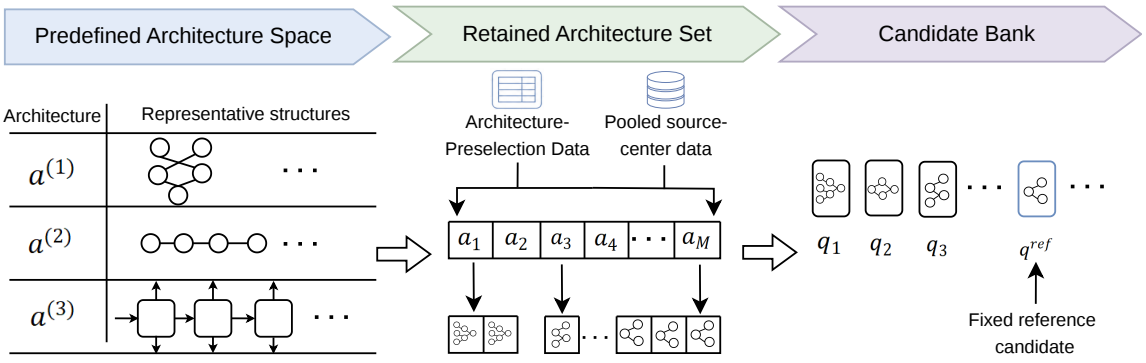



Figure 3: Candidate bank construction and reference candidate designation.

### 4.3. Target-Side Instantiation

For target center $c$, MSA-DTI computes $\mathcal{Q}_c^{\mathrm{fea}}$ using Eq. (5) and restricts target adaptation to the candidates in this set. Because the subsequent reference-based selection requires a deployment-feasible reference for comparison and fallback, MSA-DTI proceeds only if $q^{\mathrm{ref}} \in \mathcal{Q}_c^{\mathrm{fea}}$. Otherwise, it terminates without DT instantiation under the current deployment limits.

For target-center instantiation, the support set $\mathcal{D}_c^{\mathrm{sup}}$ is used for parameter adaptation, whereas the validation set $\mathcal{D}_c^{\mathrm{val}}$ is used for post-adaptation candidate comparison. For a dataset $\mathcal{D}$, let

$$\mathcal{L}(\mathcal{D}; a, \theta) = \frac{1}{|\mathcal{D}|} \sum_{(x,y)\in\mathcal{D}} \ell\left(f_{a,\theta}(x), y\right) \tag{8}$$

denote the empirical task loss.

For each feasible candidate $q = (a_q, \theta_q^0)$, let $\theta_{c,q}^{(t)}$ denote its parameters after $t$ target-side adaptation steps. With a common update budget of $T$ steps and a learning rate $\eta$ shared by all feasible candidates, the gradient update is

$$\begin{aligned} \theta_{c,q}^{(0)} &= \theta_q^0, \\ \theta_{c,q}^{(t+1)} &= \theta_{c,q}^{(t)} - \eta \nabla_\theta \mathcal{L}\left(\mathcal{D}_c^{\mathrm{sup}}; a_q, \theta_{c,q}^{(t)}\right), \\ & \qquad t = 0, \ldots, T-1. \end{aligned} \tag{9}$$

Equation (9) gives the unclipped form of the target-adaptation operator $\mathcal{U}_c$ introduced in Section 3.3. The experiments apply the gradient-norm limit in Table 1 before each update. The final parameters $\theta_{c,q}^{(T)}$ are the operator's output. After $T$ adaptation steps, let $J_c(q)$ denote the post-adaptation validation score of candidate $q$.

$$J_c(q) = \mathcal{L}\left(\mathcal{D}_c^{\mathrm{val}}; a_q, \theta_{c,q}^{(T)}\right). \tag{10}$$

Because validation-based candidate rankings are subject to estimation uncertainty, MSA-DTI uses a relative margin so that a marginal post-adaptation loss reduction does not by itself trigger replacement of the reference candidate.

Let $\mathcal{Q}_c^{\mathrm{alt}} = \mathcal{Q}_c^{\mathrm{fea}} \setminus \{q^{\mathrm{ref}}\}$ denote the feasible alternatives, and let $q_c^*$ denote the candidate selected by MSA-DTI. If $\mathcal{Q}_c^{\mathrm{alt}}$ is empty, MSA-DTI sets $q_c^* = q^{\mathrm{ref}}$ and skips alternative selection. Otherwise, let $\widetilde{q}_c \in \arg\min_{q\in\mathcal{Q}_c^{\mathrm{alt}}} J_c(q)$ denote an alternative with the lowest validation loss. Let $\tau \in [0, 1)$ denote the relative margin. The selected candidate is then

$$q_c^* = \begin{cases} \widetilde{q}_c, & J_c(\widetilde{q}_c) \le (1-\tau) J_c(q^{\mathrm{ref}}), \\ q^{\mathrm{ref}}, & \text{otherwise.} \end{cases} \tag{11}$$

A larger $\tau$ requires a larger validation-loss reduction for an alternative to be selected over the reference. The margin affects only model selection and does not change the adapted parameters.

The selected candidate $q_c^*$ provides the architecture $a_{q_c^*}$, and its adapted parameters $\theta_{c,q_c^*}^{(T)}$ define the target-specific DT model in Eq. (2). This stage outputs $\mathrm{DT}_c$. Figure 4 illustrates common-budget adaptation and the reference-based selection rule.

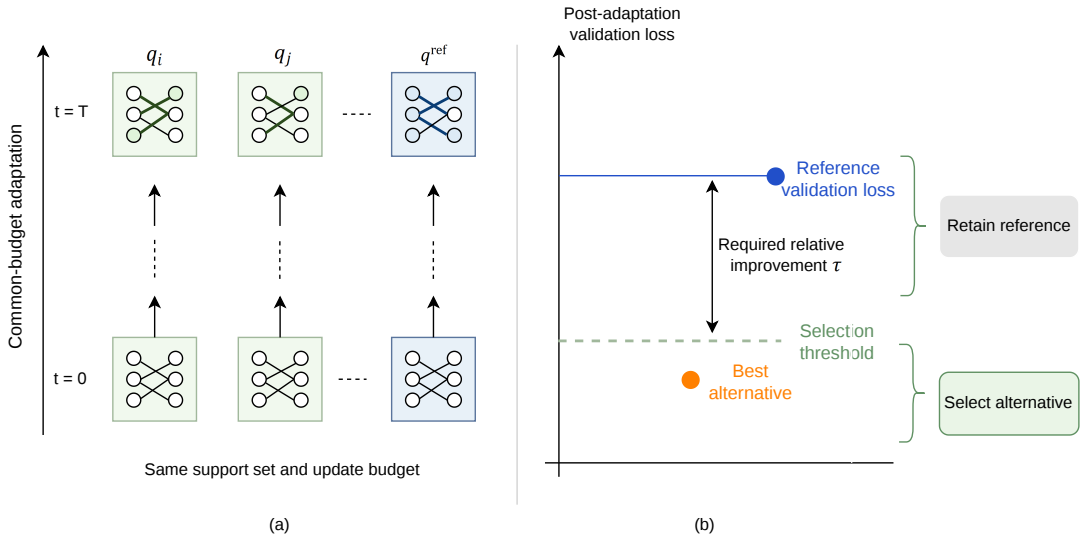



Figure 4: Common-budget adaptation and reference-based selection.

Algorithm 1 summarizes the target-side procedure when a calibrated margin is available. The reference-only fallback when calibration fails is specified in Section 4.4.

**Algorithm 1** Target-Side MSA-DTI for One Center

**Require:** Candidate bank $\mathcal{Q}$, reference candidate $q^{\text{ref}}$, support set $\mathcal{D}_c^{\text{sup}}$, validation set $\mathcal{D}_c^{\text{val}}$, deployment limits $B_c^F$ and $B_c^P$, update budget $T$, learning rate $\eta$, and relative margin $\tau$
**Ensure:** Target-specific DT model $\text{DT}_c$, if the reference candidate is deployment-feasible

Compute $\mathcal{Q}_c^{\text{fea}}$ using Eq. (5)
**if** $q^{\text{ref}} \notin \mathcal{Q}_c^{\text{fea}}$ **then**
    terminate without DT instantiation
**end if**
**for all** $q \in \mathcal{Q}_c^{\text{fea}}$ **do**
    Adapt $q$ using Eq. (9)
    Compute $J_c(q)$ using Eq. (10)
**end for**
$\mathcal{Q}_c^{\text{alt}} \leftarrow \mathcal{Q}_c^{\text{fea}} \setminus \{q^{\text{ref}}\}$
**if** $\mathcal{Q}_c^{\text{alt}} = \emptyset$ **then**
    $q_c^* \leftarrow q^{\text{ref}}$
**else**
    Choose $\widetilde{q}_c \in \arg\min_{q \in \mathcal{Q}_c^{\text{alt}}} J_c(q)$
    Select $q_c^*$ using Eq. (11)
**end if**
$\text{DT}_c \leftarrow \left(a_{q_c^*}, \theta_{c,q_c^*}^{(T)}\right)$
**return** $\text{DT}_c$

## 4.4. Relative-Margin Calibration and Analytical Properties

The margin $\tau$ is calibrated on a margin-calibration pool that is disjoint from the architecture-preselection cases and final target evaluation. Each calibration case contains disjoint support, validation, and check sets. The support and validation sets retain the roles defined in Section 4.3, whereas the check set provides independent evidence for evaluating the resulting selection during calibration. For a given margin, the harmful-selection rate is the fraction of all calibration cases in which an alternative is selected and its check-set loss exceeds that of the adapted reference.

The paired-initialization control uses a second source-trained initialization of the reference architecture and provides a same-architecture control for source-initialization effects.

Let $\mathcal{T}$ be a preset margin set. For $\tau \in \mathcal{T}$, let $\widehat{R}_{\mathrm{harm}}(\tau)$ denote the observed harmful-selection rate. Let $\widehat{G}_{\mathrm{ref}}(\tau)$ and $\widehat{G}_{\mathrm{pair}}(\tau)$ denote the mean relative check-loss gains with respect to the adapted reference and paired-initialization control, respectively, where each gain is the relative reduction in check-set loss of the selected model with respect to the corresponding control. Their lower confidence bounds are $\underline{G}_{\mathrm{ref}}(\tau)$ and $\underline{G}_{\mathrm{pair}}(\tau)$. Given a harmful-selection-rate threshold $\rho$ and mean-gain thresholds $\gamma_{\mathrm{ref}}$ and $\gamma_{\mathrm{pair}}$, the eligible margins are

$$\begin{aligned}\mathcal{T}_{\mathrm{elig}} = \big\{\tau \in \mathcal{T} \;\big|\; & \widehat{R}_{\mathrm{harm}}(\tau) \le \rho, \\ & \widehat{G}_{\mathrm{ref}}(\tau) \ge \gamma_{\mathrm{ref}}, \quad \underline{G}_{\mathrm{ref}}(\tau) > 0, \\ & \widehat{G}_{\mathrm{pair}}(\tau) \ge \gamma_{\mathrm{pair}}, \quad \underline{G}_{\mathrm{pair}}(\tau) > 0\big\}.\end{aligned} \tag{12}$$

When $\mathcal{T}_{\mathrm{elig}}$ is nonempty, the calibrated margin is the least restrictive eligible margin $\tau^* = \min \mathcal{T}_{\mathrm{elig}}$ and is fixed before final target evaluation.

If $\mathcal{T}_{\mathrm{elig}} = \varnothing$, MSA-DTI retains the adapted reference. Because the statistics in Eq. (12) are estimated from the calibration pool, the calibrated margin depends on the calibration domain.

By construction, any DT model instantiated by MSA-DTI is selected from $\mathcal{Q}_c^{\mathrm{fea}}$ and therefore satisfies both deployment limits. Moreover, whenever an alternative is selected and $J_c(q^{\mathrm{ref}}) > 0$, Eq. (11) guarantees a relative validation-loss reduction of at least $\tau$ with respect to the reference.

The target-stage computational complexity is

$$O\left(|\mathcal{Q}| + \sum_{q \in \mathcal{Q}_c^{\mathrm{fea}}} \left(T C_q^{\mathrm{grad}} + C_q^{\mathrm{val}}\right)\right). \tag{13}$$

Here, $C_q^{\mathrm{grad}}$ and $C_q^{\mathrm{val}}$ denote the costs of one target update and validation evaluation for candidate $q$, respectively. The $|\mathcal{Q}|$ term accounts for deployment filtering.

## 5. Experimental Evaluation

We evaluate prediction accuracy, selection reliability, deployment feasibility, target-side cost, and robustness across target conditions.

### 5.1. Experimental Setup

#### *5.1.1. Benchmark and Data Splits*

We use a controlled synthetic benchmark for heterogeneous multi-center workloads. The benchmark contains three center types A–C with distinct workload-dynamics ranges. Monitoring becomes progressively sparser and noisier from type A to type C. The prediction target is the next $H$ values of a normalized latent workload in $[0, 1]$; 12 simulated monitoring metrics with value/mask pairs and a time-gap feature constitute the 25-channel input. Synthetic sampling intervals are measured in simulated base steps rather than an asserted physical clock interval; MSE is evaluated on the squared workload fraction scale. Supplementary Section S8 gives the type-specific generation ranges and missingness rules. Deployment-limit tiers are platform-side settings assigned using the type-dependent probabilities specified in Supplementary Section S8.

The main study uses 20 source centers and 20 held-out target centers. Prediction horizons $H \in \{1, 4\}$ and support sizes $K \in \{10, 20\}$ give four cases per target center and 80 held-out cases in total. Here, $K$ denotes the number of support windows used for target adaptation.

For each case, the raw timeline is partitioned chronologically into disjoint support, validation, check, and test segments before window generation, and windows are constructed separately within each segment. Depending on $H$ and $K$, the validation, check, and test sets contain 1342–1351, 621–628, and 621–627 windows, respectively. Across the architecture-preselection, margin-calibration, and diagnostic pools, check sets provide the corresponding development evidence, whereas held-out target test sets are reserved for final evaluation after model selection. The source, architecture-preselection, margin-calibration, diagnostic, and held-out target pools use disjoint center identities.

### *5.1.2. Candidate Bank, Reference, and Calibration*

The initial architecture space contains 66 configurations spanning multilayer perceptron (MLP), temporal convolutional network (TCN), and gated recurrent unit (GRU) architectures. A57, a one-layer GRU with 32 hidden units, is fixed as the reference before preselection.

An alternative configuration is retained if either of the two preselection evidence counts defined in Section 4.2 is at least two. The preselection step retains six configurations corresponding to five executable architectures. Together with a second source-trained initialization of A57 used as the paired-initialization control, the resulting source-trained records form a seven-candidate bank. The reference, paired A57 control, and stronger alternative initializations do not share a single identical source-training recipe; therefore, effects of the complete bank and selector must not be interpreted as architecture-only effects.

Applying Eq. (12) to the six preset margins on 20 disjoint calibration centers (80 cases) yields the smallest eligible margin $\tau = 0.10$, which is fixed before held-out evaluation. The full five-criterion calibration grid is reported in Supplementary Section S2.2. Table 1 summarizes the experimental configuration.

Table 1: Main experimental configuration.

| Category | Item | Setting |
|---|---|---|
| Data | Centers, observations, and input length | Source/held-out = 20/20, 3000 per center, $L = 96$ |
| | Target cases and split sizes | 80 cases, $H \in \{1, 4\}$, support $K \in \{10, 20\}$, validation/check/test windows = 1342–1351 / 621–628 / 621–627 |
| Bank | Initial / retained / executable / candidates | 66/6/5/7 |
| | Reference and architecture preselection | A57, one-layer GRU-32, fixed before preselection. An alternative is retained if either preselection evidence count in Section 4.2 is $\geq 2$ |
| | Source-model records | Reference pooled-source initialization, five alternative source-trained initializations, and one additional A57 paired-initialization control |
| Training | Strong-bank alternative source training | Adam / MSE / 50 epochs, learning rate $10^{-3}$, batch size 64 |
| | MSA-DTI and fixed-reference baseline adaptation | SGD / MSE / 50 updates per adapted model, learning rate 0.01, gradient-norm limit 1.0, full support set |
| Selection | Grid and eligibility | $\{0.05, 0.075, 0.10, 0.125, 0.15, 0.20\}$, harmful-selection rate $\leq 5\%$, both mean relative gains $\geq 3\%$, both 95% lower confidence bounds $> 0$ |
| | Calibrated margin | $\tau = 0.10$ |
| Limits | Operation-count limits | Tight / medium / loose = $1.5\times10^6/5\times10^6/2\times10^7$ |
| | Parameter-count limits | Tight / medium / loose = $3\times10^4/10^5/5\times10^5$ |
| Statistics | Confidence intervals | Center-cluster bootstrap, 4000 repetitions |

### *5.1.3. Task, Baselines, and Metrics*

We use workload prediction as the evaluation task because it supports proactive resource management in computing systems [33–35]. Mean squared error (MSE) is the task loss. We also report mean absolute error (MAE), Worst-10% error, conditional value at risk at the 90th percentile (CVaR90), target-side time, and limit satisfaction.

For each case, Worst-10% error is the mean of the largest 10% of window-level MSE values on the test set. CVaR90 is the mean case-level MSE over the 10% of target cases with the largest MSE.

For the main held-out cases, beneficial and harmful alternative selections are reported against the independently evaluated pre-training and fine-tuning (PT+FT) baseline using test MSE. This differs from the within-run adapted reference used for the calibration check-set comparison. Relative gains are computed for each paired case and then averaged. The 95% confidence intervals use a center-cluster bootstrap with 4000 repetitions, resampling target centers while keeping their four cases together.

We compare MSA-DTI with PT+FT, MeDeT-based adaptation, random initialization, three task-matched NAS-inspired controls (abbreviated as few-shot NAS, zero-shot NAS, and zero-shot NAS with fine-tuning in figures and tables), and H-Meta-NAS-based adaptation [12, 30, 31]. These controls use the common architecture space with method-specific shortlisting and target evaluation; the abbreviations do not denote exact reproductions of the published NAS algorithms. PT+FT is the main controlled baseline because it uses the same source-trained reference and nominal target-adaptation settings as MSA-DTI. The remaining baselines provide method-level comparisons while retaining their method-specific initialization or search procedures.

As the closest evaluated architecture-adaptation baseline, H-Meta-NAS-based adaptation uses source-learned meta-initializations and validation-only target search after deployment filtering. Its meta-initializations and search settings are fixed before held-out evaluation, and the main comparison uses 50 SGD/MSE updates per adapted model. All methods use the same target cases, chronological splits, deployment limits, and architecture definitions.

#### *5.1.4. Runtime Protocol*

All methods are implemented in Python and PyTorch and run on an NVIDIA RTX 3060 Laptop GPU. Target-side time includes model creation, initialization loading, target-side search when applicable, target adaptation, validation evaluation, and final selection, but excludes offline preparation and test evaluation. Each reported time is the mean over five runs.

### 5.2. Overall Performance and Selection Reliability

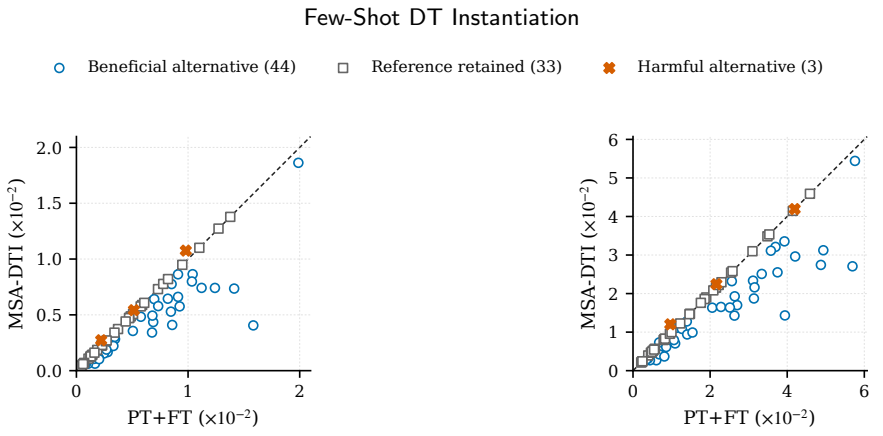
Figure 5 compares predictive performance on the held-out target cases. All selected models satisfy both deployment limits.




Figure 5: Predictive performance on the held-out target cases. Panels (a)–(d) show MAE, MSE, Worst-10% error, and CVaR90, respectively. Lower values are better.

MSA-DTI gives the lowest values for all four error measures. Compared with PT+FT, the strongest baseline, MSA-DTI reduces MAE by 8.55%, MSE by 14.60%, and Worst-10% error by 13.76% on average across paired cases. The 95% confidence interval for the paired MSE reduction is [9.41%, 20.14%]. Paired MSE comparisons with the remaining baselines also favor MSA-DTI, with each 95% confidence interval lying above zero.

Figure 6 compares MSA-DTI with PT+FT at the case level.




(a) Test MSE (b) Test Worst-10% error

Figure 6: Held-out paired comparison of MSA-DTI and PT+FT for (a) test MSE and (b) test Worst-10% error. Alternative labels use test MSE against the independently evaluated PT+FT baseline, and points below the diagonal favor MSA-DTI.

MSA-DTI retains the reference in 33 cases and selects an alternative in 47 cases. Of the 47 alternative selections, 44 have lower test MSE than the independently evaluated PT+FT baseline and three have higher test MSE. These three represent 3.75% of all held-out cases. The 95% confidence interval for this PT+FT-based harmful-alternative rate is [0.00%, 7.50%], which spans 5% and does not directly validate the within-run calibration criterion.

Because few-shot NAS and zero-shot NAS with fine-tuning use Adam and Huber loss in the main comparison, we additionally evaluate an optimizer-matched diagnostic in which the included methods use SGD, MSE, and 50 target updates per adapted model. In this diagnostic, MSA-DTI achieves the lowest MSE, Worst-10% error, and CVaR90 among the included methods. Table 2 reports these results.

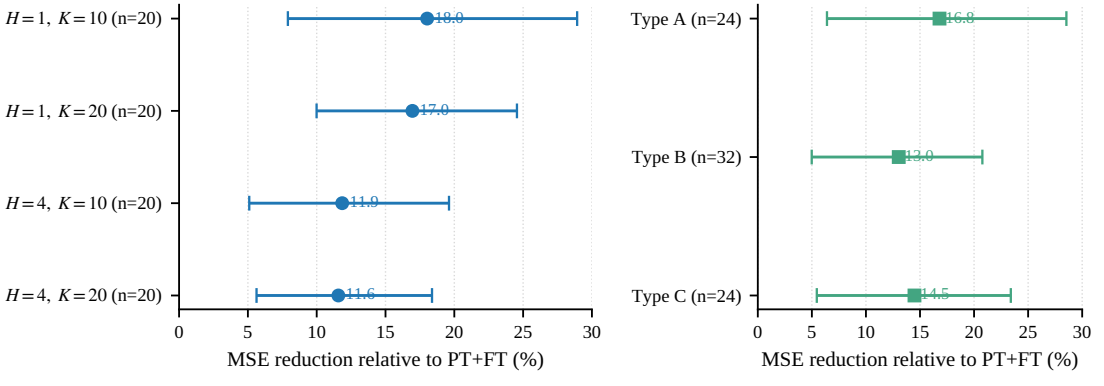
Table 2: Optimizer-matched diagnostic comparison.

| Method | MSE↓ | Worst-10%↓ | CVaR90↓ | Adapted models per case |
|---|---|---|---|---|
| PT+FT | 0.004497 | 0.017144 | 0.011878 | 1.00 |
| Few-shot NAS | 0.006715 | 0.025246 | 0.019653 | 12.00 |
| Zero-shot NAS + fine-tuning | 0.007402 | 0.028099 | 0.020402 | 12.00 |
| **MSA-DTI** | **0.003809** | **0.015094** | **0.008696** | 6.10 |

All methods use the same independent diagnostic cases, hard deployment limits, SGD optimizer, MSE loss, and 50 target updates per adapted model. The source initializations and candidate-selection rules remain method-specific.

## 5.3. Robustness and Sensitivity Analysis

Figure 7 reports paired MSE reductions relative to PT+FT. Positive values favor MSA-DTI.




(a) Horizon and support size (b) Target center type

Figure 7: Paired MSE reduction relative to PT+FT across (a) horizon/support settings and (b) target-center types. Error bars show 95% confidence intervals.

The reduction ranges from 11.57% to 18.03% across the four horizon and support-size settings and from 13.03% to 16.80% across the three center types. All seven confidence intervals remain above zero.

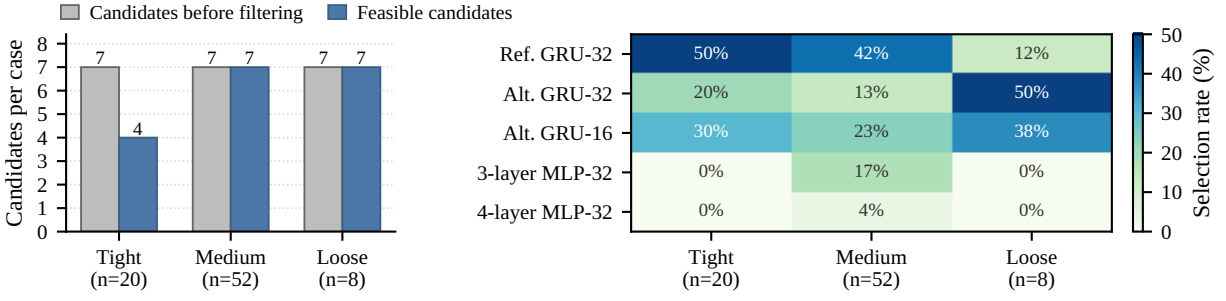
On a separate robustness pool, the fixed candidate architectures retrained under three source-training seeds yield paired MSE reductions ranging from 10.52% to 13.21%. All three 95% confidence intervals remain above zero.

Applying the same full 66-configuration architecture-preselection procedure to two new 20-center pools yields preselection-derived lists of zero and four indexed configurations, respectively, compared with six in the main-study list. The fixed A57 reference candidate is constructed separately and remains available for downstream fallback regardless of these list sizes. The two independent retained sets have Jaccard similarities of 0 and 0.429 to the main-study retained set. A separate architecture-preselection stability diagnostic evaluates the two nonempty retained sets on a fourth independent 20-center pool while keeping the downstream procedure unchanged. The 95% confidence intervals for the paired check-MSE gains relative to the reference include zero for both nonempty sets.

The retained-set variation indicates sample sensitivity in candidate discovery. To examine how this variability propagates through the complete instantiation procedure, we then construct the corresponding source-trained banks, apply the same margin-calibration rule to each bank, and evaluate the resulting pipelines on a new 20-center held-out pool. The banks derived from the main-study preselection pool and the nonempty independent preselection pool achieve mean paired MSE reductions of 10.03% (95% CI [4.60%, 16.02%]) and 8.78% (95% CI [3.83%, 14.24%]) relative to PT+FT, respectively. Each produces six harmful alternative selections, corresponding to 7.50% of the 80 held-out cases. The preselection outcome with no retained alternative therefore reverts to the adapted reference, yielding zero paired difference. All three pipelines satisfy both deployment limits in every case.

## 5.4. Deployment Feasibility and Cost

Feasibility in this subsection is assessed using the operation-count and parameter-count limits defined in Section 3.2. Figure 8 shows how the three deployment-limit settings affect candidate feasibility and final model selection.




(a) Candidate filtering

(b) Final model selection

Figure 8: Effects of the three deployment-limit settings on (a) the number of feasible candidates and (b) the selected model configurations.

Under the tight setting, four of the seven candidates in the main candidate bank are deployment-feasible, whereas all seven are feasible under the medium and loose settings.

For the main candidate bank under the tight setting, a single-limit analysis shows that the reduction is caused by the parameter-count limit. All seven candidates satisfy the operation-count limit, while the parameter-count limit excludes three MLP candidates.

To test whether the two limits independently restrict candidate feasibility, we form an eight-candidate stress bank by adding one configuration from the predefined architecture space and use diagnostic thresholds derived from the resulting model-complexity profiles. For both $H = 1$ and $H = 4$, each single-limit feasible set retains seven of eight candidates, whereas the joint feasible set retains six.

Under the joint stress limits, all models selected from the expanded bank on the same 80 held-out cases satisfy both limits. Separately, for the main candidate bank, mean paired MSE reductions relative to PT+FT remain positive under the tight, medium, and loose settings. The corresponding reductions are 11.35%, 15.30%, and 18.22%, respectively.

Table 3 compares target-side cost and selected-model complexity for PT+FT, the target-adaptation architecture-search baselines, and MSA-DTI.

Table 3: Target-side cost and selected-model complexity.

| Method | Time per case (s) | Adapted models per case | Parameter count | Estimated operation count |
|---|---|---|---|---|
| PT+FT | $0.204 \pm 0.004$ | 1.00 | 5,746 | 1,050,784 |
| Few-shot NAS | $13.471 \pm 0.173$ | 12.00 | 30,995 | 1,468,894 |
| H-Meta-NAS | $20.847 \pm 5.032$ | 12.00 | 26,627 | 1,700,279 |
| Zero-shot NAS + fine-tuning | $17.601 \pm 0.150$ | 12.00 | 62,076 | 1,571,621 |
| **MSA-DTI** | $5.676 \pm 0.059$ | 6.25 | 14,892 | 751,433 |

Time is mean ± standard deviation over five runs. Parameter counts and estimated operation counts are averaged over the final models selected in the held-out cases.

Relative to PT+FT, MSA-DTI achieves lower held-out MSE in the main evaluation but takes 5.676 rather than 0.204 seconds per target case (approximately 27.8 times, or 5.472 seconds more). It is 2.37× faster than few-shot NAS and 3.10× faster than zero-shot NAS with fine-tuning. These comparisons concern one-time target-side model instantiation, not concurrent platform scheduling. The reported target-side times are measured for one instantiation case on one GPU. Concurrent-case scaling and platform-level scheduling are outside the present evaluation.

Hardware profiling evaluates every executable architecture on an Intel Core i7-12700H CPU and an NVIDIA RTX 3060 Laptop GPU. Median batch-size-one inference latency ranges from 0.093 to 1.980 ms on the CPU and from 0.133 to 4.170 ms on the GPU. Serialized model size ranges from 10.3 to 315.9 KiB. Operation count is positively associated with median latency, whereas parameter count preserves the serialized-size ranking in all measured settings. The measured increment in peak PyTorch CUDA allocation is not monotonic in parameter count. These measurements support the two counts as distinct model-level complexity indicators. Their hardware-level implications remain host-dependent.

## 5.5. Ablation Study

Table 4 reports component variants on an independent diagnostic pool. The reference-only and two-reference-initialization variants do not reproduce the cross-architecture gain. Without the relative margin, diagnostic MSE is nearly unchanged, while the harmful-selection rate increases from 15.00% to 17.50%. The individual rate intervals overlap, but this is not a paired test of the difference. The increase is reported descriptively. Removing deployment filtering slightly lowers MSE but reduces limit satisfaction from 100% to 96.25%.

Overall, the main held-out result evaluates the complete MSA-DTI pipeline, while the diagnostic controls distinguish the roles of individual design choices. The cross-architecture gain remains sensitive to source training. The relative margin mainly affects replacement conservativeness, whereas deployment filtering enforces limit satisfaction.

Table 4: Component variants on the independent diagnostic pool.

| Variant | MSE↓ | Worst-10%↓ | Paired MSE reduction (%)↑ | Harmful-selection rate (%)↓ | Limit satisfaction (%)↑ |
|---|---|---|---|---|---|
| **MSA-DTI** | 0.004063 | 0.015764 | 15.79 | 15.00 | 100.00 |
| Earlier source training | 0.004876 | 0.019044 | 1.75 | 2.50 | 100.00 |
| Reference only | 0.005051 | 0.019512 | 0.00 | 0.00 | 100.00 |
| Two reference initializations | 0.004917 | 0.019136 | 1.29 | 1.25 | 100.00 |
| Without relative margin | 0.004055 | 0.015690 | 16.16 | 17.50 | 100.00 |
| Without deployment filtering | 0.004037 | 0.015635 | 16.13 | 13.75 | 96.25 |

Paired MSE reduction is computed relative to the adapted reference on the same diagnostic pool. Harmful-selection rate is computed over all diagnostic cases. The earlier source-training row changes the full source-training procedure for the alternatives, so it measures candidate-bank sensitivity rather than a single-component ablation.

## 5.6. Real-Workload Evaluations

The primary real-workload experiment uses the GWA-T-12 Bitbrains fastStorage trace[1] of virtual-machine resource measurements [36]. All source models are trained anew within this data domain, using 20 source VMs; separate 20-VM development and calibration pools precede a 40-VM held-out pool (160 $H/K$ cases). This is an in-domain real-data instantiation study, not zero-shot cross-dataset transfer. It predicts future recorded CPU usage from the next $H \in \{1, 4\}$ native records using $K \in \{10, 20\}$ support windows; native record gaps vary, with nominal five-minute sampling. Under the full five-criterion calibration rule, $\tau = 0.05$ is selected before held-out testing. The mean of case-relative test-MSE reductions against PT+FT is 32.70% (95% VM-cluster CI [7.67%,51.25%]); against a

[1] Dataset: https://atlarge-research.com/gwa-t-12/. The trace was provided by Bitbrains IT Services Inc.

candidate fixed by independent development VMs under the same feasibility limits, the secondary paired reduction is 38.39% (95% CI [25.56%,50.80%]). These ratios differ from the 8.09% reduction computed using the two aggregate mean MSE values. Eleven of 160 held-out cases undergo a harmful alternative selection (6.875%), exceeding the 5% empirical calibration criterion, which does not provide a held-out risk guarantee. The multi-candidate method also incurs additional target-side adaptation cost. Detailed dataset, calibration, cost, and subgroup records are provided in Supplementary Section S7.5. A separate synthetic strong-fixed control with unresolved historical development-pool provenance is reported as supplementary evidence in Section S7.6 rather than a confirmatory main result.

To examine an additional workload setting, we evaluate Alibaba Cluster Trace v2018[2] under a corrected causal-input protocol. Source models are retrained on past-only features; 20 source, 20 calibration, and 40 held-out machines yield 80 calibration and 160 held-out cases. Each model predicts $H \in \{1, 4\}$ future CPU-fraction values from 96 historical steps on a 10-second event-time grid, using $K \in \{10, 20\}$ support windows for adaptation. Observed CPU values, rather than interpolated future labels, provide the test targets. No preset margin passes the full five-criterion calibration, so MSA-DTI retains the adapted PT+FT reference throughout the held-out evaluation (0.00% paired gain). Applying the synthetic-study margin $\tau = 0.10$ only as a diagnostic yields $-0.2592\%$ mean case-relative test-MSE gain (95% machine-cluster CI $[-1.4711\%, 0.6205\%]$), without an established positive gain on this trace. This retrospective correction involves previously observed machines and assumes event timestamps represent data-availability times. Supplementary Section S7.4 provides the complete protocol, calibration table, and held-out analyses.

## 6. Conclusion

This paper presented MSA-DTI for DT model instantiation with few-shot target adaptation under per-model operation-count and parameter-count limits. MSA-DTI filters infeasible candidates from the source-trained bank, adapts the remaining candidates under a common update budget, and applies a calibrated reference-model replacement rule based on post-adaptation validation performance. In the main held-out evaluation, MSA-DTI reduces MSE by 14.60% on average across paired cases relative to PT+FT while satisfying both deployment limits in all main held-out cases. The confidence interval for the PT+FT-based held-out harmful-alternative rate spans 5%. Candidate discovery also shows sensitivity to the development pool used for architecture preselection. In a separate end-to-end analysis, the two nonempty preselection-derived banks yield positive mean paired MSE reductions relative to PT+FT with confidence intervals above zero, while the no-alternative preselection outcome reverts to the adapted reference. In a separately source-trained and calibrated Bitbrains VM cohort, MSA-DTI achieves a 32.70% mean case-relative test-MSE reduction over PT+FT; the same-bank development-fixed comparison provides secondary support. The method incurs additional target-side adaptation cost, and 11 of 160 held-out Bitbrains cases have harmful alternative selections, above the empirical 5% calibration criterion. In the causally corrected Alibaba evaluation, no preset margin qualifies, so the calibrated policy falls back to the adapted reference. These findings support positive performance on the evaluated synthetic and Bitbrains workloads without implying universal gains across real-data domains. Future work includes more stable candidate discovery, selection under domain shift, and measurement-derived deployment limits across additional platforms.

## CRediT Authorship Contribution Statement

*The final author contribution statement will be added after the author list is confirmed.*

## Funding

This work is supported by the Major Science and Technology Projects in Qinghai Province (2024-GX-A3).

## Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

[2] Alibaba Cluster Trace v2018 is available at https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018.

## Data Availability

The frozen synthetic-study code, configurations, processed results, plot-ready data, and reproduction scripts are available in GitHub Release v1.2.7 at https://github.com/w924871681/paper-code-DT-Twin/releases/tag/v1.2.7. Saved predictions and targets, selection records, and statistical verification scripts for the supplementary R5 fixed-candidate comparison and the Bitbrains in-domain evaluation are available in a separate evidence-only Release at https://github.com/w924871681/paper-code-DT-Twin/releases/tag/evidence-r5-bitbrains-20261013. This evidence archive permits recalculation of the saved results but does not include the original Bitbrains traces or source-model checkpoints. The causally corrected Alibaba study is documented in a separate reproducibility archive, `Alibaba_Causal_Correction_v1_1_G5_COMPLETE_20261011.zip`, containing the corrected code, model checkpoints, derived results, and provenance records; this archive is not included in either public Release and has not yet been publicly deposited. The original Alibaba Cluster Trace v2018 and Bitbrains GWA-T-12 traces are not redistributed; their official access links are provided in the corresponding dataset footnotes.

## Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the authors used OpenAI ChatGPT to assist with language refinement, manuscript organization, and readability improvement. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

## References


[1] L. U. Khan, Z. Han, W. Saad, E. Hossain, M. Guizani, and C. S. Hong, "Digital twin of wireless systems: Overview, taxonomy, challenges, and opportunities," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 4, pp. 2230–2254, 4th Quart. 2022, doi: 10.1109/COMST.2022.3198273.

[2] S. Mihai et al., "Digital twins: A survey on enabling technologies, challenges, trends and future prospects," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 4, pp. 2255–2291, 4th Quart. 2022, doi: 10.1109/COMST.2022.3208773.

[3] H. Xu, J. Wu, Q. Pan, X. Guan, and M. Guizani, "A survey on digital twin for Industrial Internet of Things: Applications, technologies and tools," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 2569–2598, 4th Quart. 2023, doi: 10.1109/COMST.2023.3297395.

[4] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, 2019, doi: 10.1109/TII.2018.2873186.

[5] J. Li et al., "Digital twin-enabled service provisioning in edge computing via continual learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 6, pp. 7335–7350, 2024, doi: 10.1109/TMC.2023.3332668.

[6] H. Sartaj, S. Ali, and J. M. Gjøby, "MeDeT: Medical device digital twins creation with few-shot meta-learning," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 6, Art. no. 158, pp. 1–36, 2025, doi: 10.1145/3708534.

[7] A. Vettoruzzo, M.-R. Bouguelia, J. Vanschoren, T. Rögnvaldsson, and K. C. Santosh, "Advances and challenges in meta-learning: A technical review," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 7, pp. 4763–4779, 2024, doi: 10.1109/TPAMI.2024.3357847.

[8] H. Gharoun, F. Momenifar, F. Chen, and A. H. Gandomi, "Meta-learning approaches for few-shot learning: A survey of recent advances," *ACM Comput. Surv.*, vol. 56, no. 12, Art. no. 294, pp. 1–41, 2024, doi: 10.1145/3659943.

[9] M. Tan et al., "MnasNet: Platform-aware neural architecture search for mobile," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 2820–2828, doi: 10.1109/CVPR.2019.00293.

[10] B. Wu et al., "FBNet: Hardware-aware efficient ConvNet design via differentiable neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 10726–10734, doi: 10.1109/CVPR.2019.01099.

[11] Y. Zhao, L. Wang, and T. Guo, "Multi-objective neural architecture search by learning search space partitions," *J. Mach. Learn. Res.*, vol. 25, no. 177, pp. 1–41, 2024.

[12] Y. Zhao, X. Gao, I. Shumailov, N. Fusi, and R. Mullins, "Rapid model architecture adaption for meta-learning," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 18721–18732, doi: 10.52202/068431-1360.

[13] D. Lian et al., "Towards fast adaptation of neural architectures with meta learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[14] P. Eustratiadis, L. Dudziak, D. Li, and T. Hospedales, "Neural fine-tuning search for few-shot learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[15] B. Qin et al., "Machine and deep learning for digital twin networks: A survey," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34694–34716, Nov. 2024, doi: 10.1109/JIOT.2024.3416733.

[16] A. Hakiri, A. Gokhale, S. Ben Yahia, and N. Mellouli, "A comprehensive survey on digital twin for future networks and emerging Internet of Things industry," *Comput. Netw.*, vol. 244, Art. no. 110350, 2024, doi: 10.1016/j.comnet.2024.110350.

[17] Y. Wu, K. Zhang, and Y. Zhang, "Digital twin networks: A survey," *IEEE Internet Things J.*, vol. 8, no. 18, pp. 13789–13804, Sep. 2021, doi: 10.1109/JIOT.2021.3079510.

[18] M. Emami Khansari and S. Sharifian, "A deep reinforcement learning approach towards distributed Function as a Service (FaaS) based edge application orchestration in cloud-edge continuum," *J. Netw. Comput. Appl.*, vol. 233, Art. no. 104042, 2025, doi: 10.1016/j.jnca.2024.104042.

[19] D. Hortelano et al., "A comprehensive survey on reinforcement-learning-based computation offloading techniques in Edge Computing Systems," *J. Netw. Comput. Appl.*, vol. 216, Art. no. 103669, 2023, doi: 10.1016/j.jnca.2023.103669.

[20] J. Son, S. Lee, and G. Kim, "When meta-learning meets online and continual learning: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 1, pp. 413–432, 2025, doi: 10.1109/TPAMI.2024.3463709.

[21] L. M. Zintgraf, K. Shiarlis, V. Kurin, K. Hofmann, and S. Whiteson, "Fast context adaptation via meta-learning," in *Proc. 36th Int. Conf. Mach. Learn. (ICML)*, vol. 97, 2019, pp. 7693–7702.

[22] Y. Zhao, T. Zhang, J. Li, and Y. Tian, "Dual adaptive representation alignment for cross-domain few-shot learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 10, pp. 11720–11732, 2023, doi: 10.1109/TPAMI.2023.3272697.

[23] J. Wu, X. Liu, X. Yin, T. Zhang, and Y. Zhang, "Task-adaptive prompted transformer for cross-domain few-shot learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 6, pp. 6012–6020, 2024, doi: 10.1609/aaai.v38i6.28416.

[24] W. Wang, L. Duan, Y. Wang, J. Fan, and Z. Zhang, "MMT: Cross domain few-shot learning via meta-memory transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 12, pp. 15018–15035, Dec. 2023, doi: 10.1109/TPAMI.2023.3306352.

[25] Z. Zuo, Y. Huang, Z. Li, Y. Jiang, and C. Liu, "Mixed contrastive transfer learning for few-shot workload prediction in the cloud," *Computing*, vol. 107, no. 1, Art. no. 5, 2025, doi: 10.1007/s00607-024-01366-y.

[26] J. Bornschein, F. Visin, and S. Osindero, "Small data, big decisions: Model selection in the small-data regime," in *Proc. 37th Int. Conf. Mach. Learn. (ICML)*, vol. 119, 2020, pp. 1035–1044.

[27] A. Lacoste, M. Marchand, F. Laviolette, and H. Larochelle, "Agnostic Bayesian learning of ensembles," in *Proc. 31st Int. Conf. Mach. Learn. (ICML)*, vol. 32, 2014, pp. 611–619.

[28] K. You, Y. Liu, J. Wang, and M. Long, "LogME: Practical assessment of pre-trained models for transfer learning," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139, 2021, pp. 12133–12143.

[29] K. You, Y. Liu, Z. Zhang, J. Wang, M. I. Jordan, and M. Long, "Ranking and tuning pre-trained models: A new paradigm for exploiting model hubs," *J. Mach. Learn. Res.*, vol. 23, no. 209, pp. 1–47, 2022.

[30] Y. Zhao, L. Wang, Y. Tian, R. Fonseca, and T. Guo, "Few-shot neural architecture search," in *Proc. 38th Int. Conf. Mach. Learn. (ICML)*, vol. 139, 2021, pp. 12707–12718.

[31] G. Li, D. Hoang, K. Bhardwaj, M. Lin, Z. Wang, and R. Marculescu, "Zero-shot neural architecture search: Challenges, solutions, and opportunities," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 12, pp. 7618–7635, Dec. 2024, doi: 10.1109/TPAMI.2024.3395423.

[32] M.-T. Wu, H.-I. Lin, and C.-W. Tsai, "A training-free neural architecture search algorithm based on search economics," *IEEE Trans. Evol. Comput.*, vol. 28, no. 2, pp. 445–459, Apr. 2024, doi: 10.1109/TEVC.2023.3264533.

[33] Y.-M. Kim, S. Song, B.-M. Koo, J. Son, Y. Lee, and J.-G. Baek, "Enhancing long-term cloud workload forecasting framework: Anomaly handling and ensemble learning in multivariate time series," *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 789–799, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3400859.

[34] F. Zhao, W. Lin, S. Lin, S. Tang, and K. Li, "MSCNet: Multi-scale network with convolutions for long-term cloud workload prediction," *IEEE Trans. Serv. Comput.*, vol. 18, no. 2, pp. 969–982, Mar.–Apr. 2025, doi: 10.1109/TSC.2025.3536313.

[35] B. Feng and Z. Ding, "Application-oriented cloud workload prediction: A survey and new perspectives," *Tsinghua Sci. Technol.*, vol. 30, no. 1, pp. 34–54, Feb. 2025, doi: 10.26599/TST.2024.9010024.

[36] S. Shen, V. van Beek, and A. Iosup, "Statistical characterization of business-critical workloads hosted in cloud datacenters," in *Proc. 15th IEEE/ACM Int. Symp. Cluster, Cloud and Grid Comput. (CCGrid)*, 2015, pp. 465–474, doi: 10.1109/CCGrid.2015.60.